

Data Structures & C Programming Notes

Classification of Data Structures

- **Primitive Data Structures**: int, float, char, boolean (basic types)
- **Non-Primitive Data Structures**:
 - **Linear**: Array, List, Stack, Queue
 - **Non-Linear**: Tree, Graph

Elementary Data Organization

- **Bit** → Byte → Field → Record → File → Database
- Example: Student data (RollNo, Name, Age → Record; multiple records → File; multiple files → Database)

Time and Space Complexity

- **Time Complexity**: Time taken by algorithm (Big-O)
 - Linear Search $O(n)$, Binary Search $O(\log n)$, Bubble Sort $O(n^2)$, Merge Sort $O(n \log n)$
- **Space Complexity**: Memory used by algorithm
 - Iterative Fibonacci $O(1)$, Recursive Fibonacci $O(n)$

String Processing in C (functions)

- strlen(), strcpy(), strcat(), strcmp(), strcmp()

Dynamic Memory Allocation (DMA)

- **Static Allocation**: Done at compile time, fixed size.
- **Dynamic Allocation**: Done at runtime, flexible.

Memory Allocation Functions

- malloc() → allocates block (garbage values)
- calloc() → allocates multiple blocks (initialized to 0)
- realloc() → resizes allocated memory
- free() → deallocates memory

Accessing Address of a Variable

- Use `&` operator to get address.
- Example: `printf("%p", &a);`

Declaring and Initializing Pointers

- Declaration: `int *p;`
- Initialization: `p = &a;`
- Dereferencing: `*p` gives value at address.

Accessing Variable through Pointer

- Example:

```
``c
int a = 20;
int *p = &a;
printf("%d", *p); // prints value of a
*p = 50; // updates a
```
```

## Recursion

- Function calling itself with a base condition.

## Advantages

- Simplifies problems, improves readability, fits mathematical definitions.

## Recursive Programs

- **Binomial Coefficient (nCr)**:

$C(n,r) = C(n-1,r-1) + C(n-1,r)$ , base cases  $C(n,0) = C(n,n) = 1$

- **Fibonacci**:

$F(0)=0, F(1)=1, F(n)=F(n-1)+F(n-2)$

- **GCD (Euclid's Algorithm)**:

$GCD(a,b) = GCD(b, a \% b)$ , base case  $GCD(a,0) = a$